

Statment 1

PROJECT OVERVIEW :

The Simple ATM Simulator is a console-based application designed to mimic core Automated Teller Machine banking operations. Developed using foundational programming building blocks, it highlights the practical application of conditional control flows (if-elif-else) and infinite execution loops (while True) to manage persistent, continuous user sessions. The system provides a secure environment for basic transaction options, ensuring the active session loops indefinitely until explicitly terminated by the user.

INTRODUCTION:

Automated Teller Machines (ATMs) revolutionized the banking sector by providing customers with 24/7 access to financial transactions without human tellers. At the heart of software systems running modern ATMs are concepts of state verification, sequence loops, and condition-based decision making.

This project explores how a terminal-based simulation can mirror this complex infrastructure. By using a conditional loop, the application keeps the system running continuously across multiple actions. Within this loop, conditional logic statements evaluate user variables—such as matching a secure PIN, parsing menu choices, verifying positive inputs, and checking for insufficient funds—to securely update data parameters and manage the application's lifecycle.

PROJECT TOPIC SUBTITLES:

1) PROJECT SYNOPSIS AND SCOPE:

The Project Synopsis & Scope serves as the project's foundation. It defines exactly what the program does, how it works under the hood, and what its limitations are:

1.1) Definition of a Terminal-Based ATM Simulator:

This defines the project as a text-only, command-line interface (CLI) program. Instead of using a physical card reader or touch screen, it mimics real banking hardware using keyboard inputs and text menus. It is designed to demonstrate core backend programming logic—specifically conditional logic and infinite loops—in a safe, simulated environment.

1.2) Core Objectives and Functional Boundaries:

This outlines what the application can and cannot do. The objective is to handle basic transactions like PIN validation, balance checks, deposits, and withdrawals securely. The functional boundaries (or scope limitations) clarify that this is a single-session simulation; it handles one user at a time, uses pre-set data, and does not connect to actual bank networks or external databases.

1.3) Hardware and Software Environment Requirements:

This lists the technical setup needed to run the program. Because it is a simple console script, hardware demands are virtually zero (any standard PC can run it). The software requirements typically just include a code execution environment—such as a Python 3.x interpreter, a C++ compiler, or a Java Runtime Environment (JRE)—along with a standard terminal window.

2) CONTROL FLOW AND FOUNDATION CONCEPTS :

This section explains the core programming logic that keeps the ATM simulator running and allows it to make decisions based on user input.

2.1)Mechanics of Infinite Iteration Loops (while True):

An ATM cannot simply turn off after you complete a single action; it must remain active to process multiple requests. An infinite loop keeps the application continuously running in the background. It repeatedly displays

the transaction menu after every operation until the user explicitly decides to quit.

2.2) Application of Breaking Constraints (break and exit()):

Because the loop is designed to run forever, the program needs structural escape hatches. The break statement is used to snap out of the current loop (for example, moving past the PIN screen once authentication is successful, or shutting down the transaction menu). The exit() function completely terminates the entire program when the user chooses to walk away.

2.3) Implementing Multi-Branch Conditionals (if-elif-else):

This structure functions as the ATM's decision-making matrix. When a user inputs a number from the menu (e.g., 1, 2, or 3), the program evaluates this choice down different pathways. if handles the first choice, elif (else-if) checks the subsequent choices, and else catches any invalid inputs (like typing a letter or an incorrect number) to prevent the system from crashing.

2.4)Nested Logic Blocks for Transaction Checks:

Nesting means placing one conditional statement inside another. For instance, when a user requests a cash withdrawal, the program first enters the withdrawal module. Inside that module, a second, nested if statement checks whether the requested amount is greater than the available account balance. This ensures transactions are only processed if strict safety criteria are met.